

AGE(1)

AGE(1)

NAME**age** - simple, modern, and secure file encryption**SYNOPSIS**

```
age [--encrypt] (-r RECIPIENT | -R PATH)... [--armor] [-o OUTPUT] [INPUT]
age [--encrypt] --passphrase [--armor] [-o OUTPUT] [INPUT]
age --decrypt [-i PATH | -j PLUGIN]... [-o OUTPUT] [INPUT]
```

DESCRIPTION

age encrypts or decrypts *INPUT* to *OUTPUT*. The *INPUT* argument is optional and defaults to standard input. Only a single *INPUT* file may be specified. If **-o** is not specified, *OUTPUT* defaults to standard output.

If **-p/--passphrase** is specified, the file is encrypted with a passphrase requested interactively. Otherwise, it's encrypted to one or more [RECIPIENTS](#) specified with **-r/--recipient** or **-R/--recipients-file**. Every recipient can decrypt the file.

In **-d/--decrypt** mode, passphrase-encrypted files are detected automatically and the passphrase is requested interactively. Otherwise, one or more [IDENTITIES](#) specified with **-i/--identity** are used to decrypt the file.

age encrypted files are binary and not malleable, with around 200 bytes of overhead per recipient, plus 16 bytes every 64KiB of plaintext.

OPTIONS

-o, --output=OUTPUT

Write encrypted or decrypted file to *OUTPUT* instead of standard output. If *OUTPUT* already exists it will be overwritten. If encrypting without **--armor**, **age** will refuse to output binary to a TTY. This can be forced by specifying **-** as *OUTPUT*.

--version

Print the version and exit.

Encryption options

-e, --encrypt

Encrypt *INPUT* to *OUTPUT*. This is the default.

-r, --recipient=RECIPIENT

Encrypt to the explicitly specified *RECIPIENT*. See the [RECIPIENTS AND IDENTITIES](#) section for possible recipient formats. This option can be repeated and combined with other recipient flags, and the file can be decrypted by all provided recipients independently.

-R, --recipients-file=PATH

Encrypt to the [RECIPIENTS](#) listed in the file at *PATH*, one per line. Empty lines and lines starting with **#** are ignored as comments. If *PATH* is **-**, the recipients are read from standard input. In this case, the *INPUT* argument must be specified.

This option can be repeated and combined with other recipient flags, and the file can be decrypted by all provided recipients independently.

-p, --passphrase

Encrypt with a passphrase, requested interactively from the terminal. **age** will offer to auto-generate a secure passphrase. This option can't be used with other recipient flags.

-a, --armor

Encrypt to an ASCII-only "armored" encoding. **age** armor is a strict version of PEM with type **AGE ENCRYPTED FILE**, canonical "strict" Base64, no headers, and no support for leading and trailing extra data.

Decryption transparently detects and decodes ASCII armoring.

-i, --identity=PATH

Encrypt to the [RECIPIENTS](#) corresponding to the [IDENTITIES](#) listed in the file at *PATH*. This is equivalent to converting the file at *PATH* to a recipients file with **age-keygen -y** and then passing that to **-R/--recipients-file**. For the format of *PATH*, see the definition of **-i/--identity** in the [Decryption options](#) section.

-e/--encrypt must be explicitly specified when using **-i/--identity** in encryption mode to avoid confusion.

-j PLUGIN

Encrypt using the data-less [plugin PLUGIN](#).

This is equivalent to using **-i/--identity** with a file that contains a single plugin **IDENTITY** that encodes no plugin-specific data.

-e/--encrypt must be explicitly specified when using **-j** in encryption mode to avoid confusion.

Decryption options

-d, --decrypt

Decrypt *INPUT* to *OUTPUT*.

If *INPUT* is passphrase encrypted, it will be automatically detected and the passphrase will be requested interactively. Otherwise, the [IDENTITIES](#) specified with **-i/--identity** are used.

ASCII armoring is transparently detected and decoded.

-i, --identity=PATH

Decrypt using the [IDENTITIES](#) at *PATH*.

PATH may be one of the following:

a. A file listing [IDENTITIES](#) one per line. Empty lines and lines starting with **"#"** are ignored as comments.

b. A passphrase encrypted age file, containing [IDENTITIES](#) one per line like above. The passphrase is requested interactively. Note that passphrase-protected identity files are not necessary for most use cases, where access to the encrypted identity file implies access to the whole system.

c. An SSH private key file, in PKCS#1, PKCS#8, or OpenSSH format. If the private key is password-protected, the password is requested interactively only if the SSH identity matches the file. See the [SSH keys](#) section for more information, including supported key types.

d. **"-"**, causing one of the options above to be read from standard input. In this case, the *INPUT* argument must be specified.

This option can be repeated. Identities are tried in the order in which are provided, and the first one matching one of the file's recipients is used. Unused identities are ignored, but it is an error if the *INPUT* file is passphrase-encrypted and **-i/--identity** is specified.

-j PLUGIN

Decrypt using the data-less [plugin PLUGIN](#).

This is equivalent to using **-i/--identity** with a file that contains a single plugin **IDENTITY** that encodes no plugin-specific data.

RECIPIENTS AND IDENTITIES

RECIPIENTS are public values, like a public key, that a file can be encrypted to. **IDENTITIES** are private values, like a private key, that allow decrypting a file encrypted to the corresponding **RECIPIENT**.

Native keys

Native **age** key pairs are generated with [age-keygen\(1\)](#), and provide small encodings and strong encryption based on X25519 for classic keys, and X25519 + ML-KEM-768 for post-quantum hybrid keys. The post-quantum hybrid keys are secure against future quantum computers and are the recommended recipient type for most applications.

A hybrid **RECIPIENT** encoding begins with **age1pq1** and looks like the following:

```
age1pq167[... 1950 more characters ...]
```

A hybrid **IDENTITY** encoding begins with **AGE-SECRET-KEY-PQ-1** and looks like the following:

```
AGE-SECRET-KEY-PQ-1K30MYPZAHAXHR22YHH27EGDVLU0QNSUH3DSV7J7NR3X6D9LHXNWSDLTV4T
```

A classic **RECIPIENT** encoding begins with **age1** and looks like the following:

```
age1gde3ncmah1qd9gg50tan199r960llztrhfapnmx853s4tjum03uqfssgdh
```

A classic **IDENTITY** encoding begins with **AGE-SECRET-KEY-1** and looks like the following:

```
AGE-SECRET-KEY-1KTyK6RVLN5TAPE7VF6FQQSKZ9HWWCDKUGXXNUQDWZ7XXT5YK5LSF3UTKQ
```

A file can't be encrypted to both post-quantum and classic keys, as that would defeat the post-quantum security of the encryption.

An encrypted file can't be linked to the native recipient it's encrypted to without access to the corresponding identity.

SSH keys

As a convenience feature, **age** also supports encrypting to RSA or Ed25519 ssh(1) keys. RSA keys must be at least 2048 bits. This feature employs more complex cryptography, and should only be used when a native key is not available for the recipient. Note that SSH keys might not be protected long-term by the recipient, since they are revokable when used only for authentication.

A **RECIPIENT** encoding is an SSH public key in **authorized_keys** format (see the **AUTHORIZED_KEYS FILE FORMAT** section of sshd(8)), starting with **ssh-rsa** or **ssh-ed25519**, like the following:

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDULTit0KUehbi[...]GU4BtElAbzh8=
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIH9p05pz22JZEas[...]l1uZc31FGYMXa
```

The comment at the end of the line, if present, is ignored.

In recipient files passed to **-R/--recipients-file**, unsupported but valid SSH public keys are ignored with a warning, to facilitate using **authorized_keys** or GitHub **.keys** files. (See [EXAMPLES](#).)

An **IDENTITY** is an SSH private key *file* passed individually to **-i/--identity**. Note that keys held on hardware tokens such as YubiKeys or accessed via ssh-agent(1) are not supported.

An encrypted file *can* be linked to the SSH public key it was encrypted to. This is so that **age** can identify the correct SSH private key before requesting its password, if any.

Plugins

age can be extended through plugins. A plugin is only loaded if a corresponding **RECIPIENT** or **IDENTITY** is specified. (Simply decrypting a file encrypted with a plugin will not cause it to load, for security reasons among others.)

A **RECIPIENT** for a plugin named **example** starts with **age1example1**, while an **IDENTITY** starts with **AGE-PLUGIN-EXAMPLE-1**. They both encode arbitrary plugin-specific data, and are generated by the plugin.

When either is specified, **age** searches for **age-plugin-example** in the PATH and executes it to perform the file header encryption or decryption. The plugin may request input from the user through **age** to complete the operation.

Plugins can be freely mixed with other plugins or natively supported keys.

A plugin is not bound to only encrypt or decrypt files meant for or generated by the plugin. For example, a plugin can be used to decrypt files encrypted to a native X25519 **RECIPIENT** or even with a passphrase. Similarly, a plugin can encrypt a file such that it can be decrypted without the use of any plugin.

Plugins for which the **IDENTITY/RECIPIENT** distinction doesn't make sense (such as a symmetric encryption plugin) may generate only an **IDENTITY** and instruct the user to perform encryption with the **-e/--encrypt** and **-i/--identity** flags. Plugins for which the concept of separate identities doesn't make sense (such as a password-encryption plugin) may instruct the user to use the **-j** flag.

Tagged recipients

age can natively encrypt to recipients starting with **age1tag1** (using P-256 ECDH) or **age1tagpq1** (using the ML-KEM-768 + P-256 post-quantum hybrid). These are intended to be the public side of private keys held in hardware.

They are directly supported to avoid the need to install the plugin, which may be platform-specific, on the encrypting side.

The tag reduces privacy, by allowing an observer to correlate files with a recipient (but not files amongst them without knowledge of the recipient), but

this is also a desirable property for hardware keys that require user interaction for each decryption operation.

EXIT STATUS

age will exit 0 if and only if encryption or decryption are successful for the full length of the input.

If an error occurs during decryption, partial output might still be generated, but only if it was possible to securely authenticate it. No unauthenticated output is ever released.

BACKWARDS COMPATIBILITY

Files encrypted with a stable version (not alpha, beta, or release candidate) of **age**, or with any v1.0.0 beta or release candidate, will decrypt with any later version of the tool.

If decrypting older files poses a security risk, doing so might cause an error by default. In this case, a flag will be provided to force the operation.

EXAMPLES

Generate a new post-quantum identity, encrypt data, and decrypt:

```
$ age-keygen -pq -o key.txt
Public key: age1pq167[...] 1950 more characters ...]

$ tar cvz ~/data | age -r age1pq167[...] > data.tar.gz.age

$ age -d -o data.tar.gz -i key.txt data.tar.gz.age
```

Encrypt **example.jpg** to multiple recipients and output to **example.jpg.age**:

```
$ age -o example.jpg.age -r age1pq167[...] -r age1pq1e3[...] example.jpg
```

Encrypt to a list of recipients:

```
$ cat > recipients.txt
# Alice
age1pq167[...] 1950 more characters ...]
# Bob
age1pq1e3[...] 1950 more characters ...]

$ age -R recipients.txt example.jpg > example.jpg.age
```

Encrypt and decrypt a file using a passphrase:

```
$ age -p secrets.txt > secrets.txt.age
Enter passphrase (leave empty to autogenerate a secure one):
Using the autogenerated passphrase "release-response-step-brand-wrap-ankle-pair-unusual-sword-train".

$ age -d secrets.txt.age > secrets.txt
Enter passphrase:
```

Encrypt and decrypt with a passphrase-protected identity file:

```
$ age-keygen | age -p > key.age
Public key: age1yhm4gctwfmrpz87tdslm550wrx6m79y9f2hdzt0lndjnehwj0ukqrjpyx5
Enter passphrase (leave empty to autogenerate a secure one):
Using the autogenerated passphrase "hip-roast-boring-snake-mention-east-wasp-honey-input-actress".

$ age -r age1yhm4gctwfmrpz87tdslm550wrx6m79y9f2hdzt0lndjnehwj0ukqrjpyx5 secrets.txt > secrets.txt.age

$ age -d -i key.age secrets.txt.age > secrets.txt
Enter passphrase for identity file "key.age":
```

Encrypt and decrypt with an SSH public key:

```
$ age -R ~/.ssh/id_ed25519.pub example.jpg > example.jpg.age

$ age -d -i ~/.ssh/id_ed25519 example.jpg.age > example.jpg
```

Encrypt and decrypt with age-plugin-yubikey:

```
$ age-plugin-yubikey # run interactive setup, generate identity file and obtain recipient

$ age -r age1yubikey1qwt50d05nh5vutpdzmlg5wn80xq5negm4uj9ghv0snvdd3yysf5yw3rhl3t secrets.txt > secrets.txt.age
```

```
$ age -d -i age-yubikey-identity-388178f3.txt secrets.txt.age
```

Encrypt to the SSH keys of a GitHub user:

```
$ curl https://github.com/benjojo.keys | age -R - example.jpg > example.jpg.age
```

SEE ALSO

[age-keygen\(1\)](#), [age-inspect\(1\)](#).

AUTHORS

Filippo Valsorda age@filippo.io

AUGUST 2026

AGE(1)